CC4P1 Programación Concurrente y Distribuida · Parcial 2026-II

Open AI Cubes

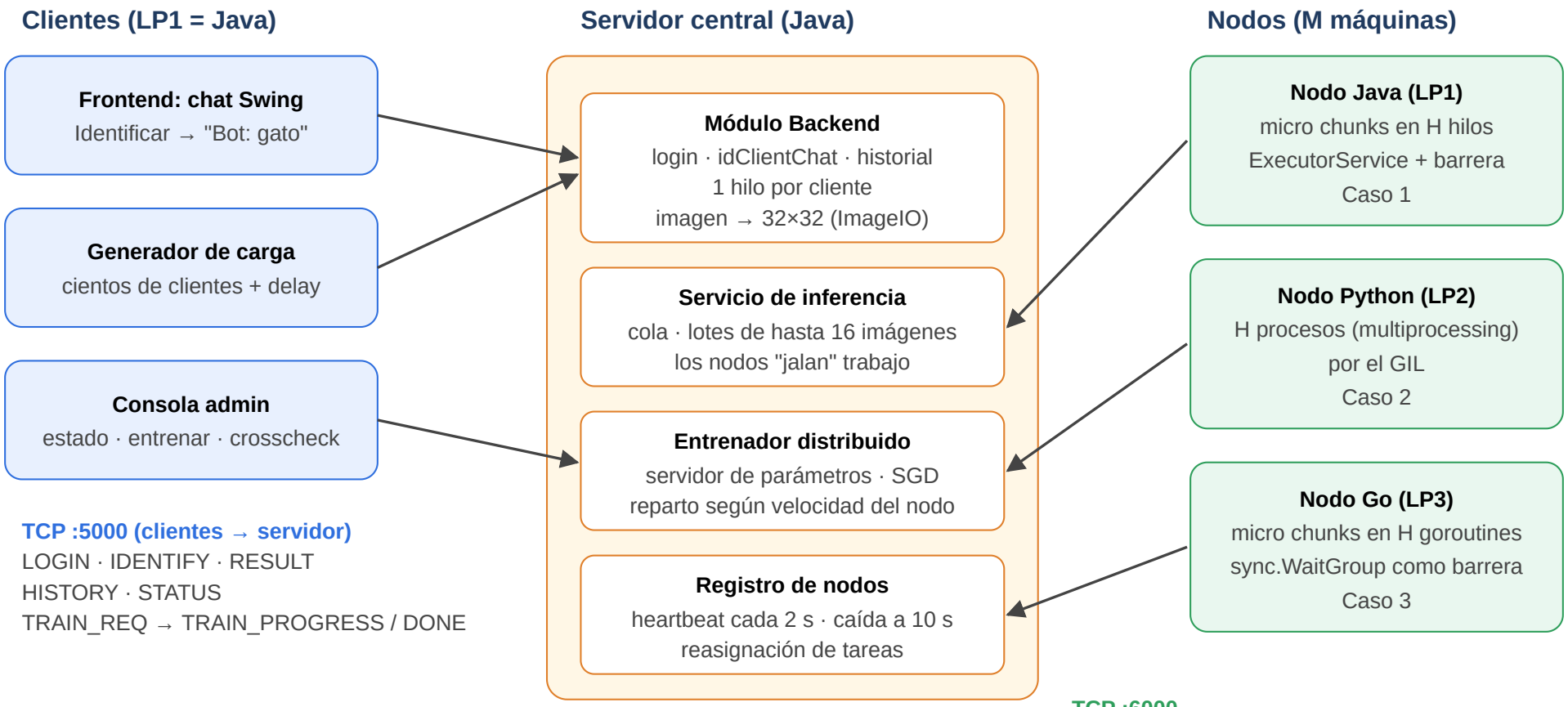
CNN concurrente, paralela y distribuida con nodos en Java, Python y Go

[Integrante 1] · [Integrante 2] · [Integrante 3]

Qué construimos

- El cubo de **procesamiento de imágenes**: una CNN (convoluciones + MLP) que se entrena y responde consultas en varios nodos
- **Backend** que atiende consultas de chat y **frontend de escritorio** (Swing): "Yo: Identificar" → "Bot: gato"
- Solo librería estándar: sockets TCP, hilos, procesos, goroutines. Sin HTTP, WebSocket ni middleware
- Clientes y servidor en **Java (JDK 8+)**; nodos en **Java, Python y Go** (Casos 1, 2 y 3)
- Evaluado en **12 casos**: 4 escenarios × 3 casos de lenguaje, más pruebas de saturación

Arquitectura



Las flechas indican quién abre la conexión: clientes y nodos se conectan al servidor, así el servidor no necesita conocer de antemano las IP de los nodos.

Protocolo binario OC/1 sobre sockets estándar: cabecera de 12 bytes ('O','C', versión, tipo, id, longitud) + payload big-endian.

Hosts y clientes

- **Servidor central** (1 máquina, Java)

  puerto 5000 clientes · 6000 nodos

  backend + coordinador + servidor de parámetros
- **Nodos** (M máquinas)

  se conectan solos al servidor · solo calculan

  Java, Python o Go · H hilos
- **Chat Swing**

  login, idClientChat, Identificar, historial
- **Generador de carga**

  cientos de clientes con delay aleatorio
- **Consola admin**

  estado · entrenar · prueba cruzada

La CNN

Capa	Salida
Entrada (CIFAR-10, 4 clases)	3×32×32
conv 3×3 ×16 + ReLU → pool	16×16×16
conv 3×3 ×32 + ReLU → pool	32×8×8
MLP 2048 → 128 + ReLU	128
MLP 128 → 4 + softmax	4

82,3 %

de acierto en 4 000 imágenes de prueba

avión · automóvil · gato · barco

267 876 parámetros · SGD con momentum

Micro chunks

- **Conv chunk**: rango de filtros (forward y gradiente de pesos) o de canales (gradiente de entrada)
- **MLP chunk**: rango de neuronas, o de entradas en el backward
- **Pool chunk**: rango de canales
- Cada rango escribe en su propia zona de memoria → **sin candados**
- Barrera por capa: la capa siguiente espera la salida completa
- Java: `ExecutorService` · Go: goroutines + `WaitGroup` · Python: `multiprocessing` (GIL)

Qué se paraleliza y qué se distribuye

Nivel	Qué se reparte
Distribuido (máquinas)	Las imágenes de cada lote; cada nodo devuelve la suma de sus gradientes
Paralelo (hilos)	Los micro chunks de cada capa: filtros, neuronas, canales
Concurrente (servidor)	Muchos clientes y nodos a la vez: hilo por conexión + cola

- Reparto proporcional a la **velocidad medida** de cada nodo (Python recibe menos)
- No se reparten capas entre máquinas: habría que mandar activaciones por la red en cada capa

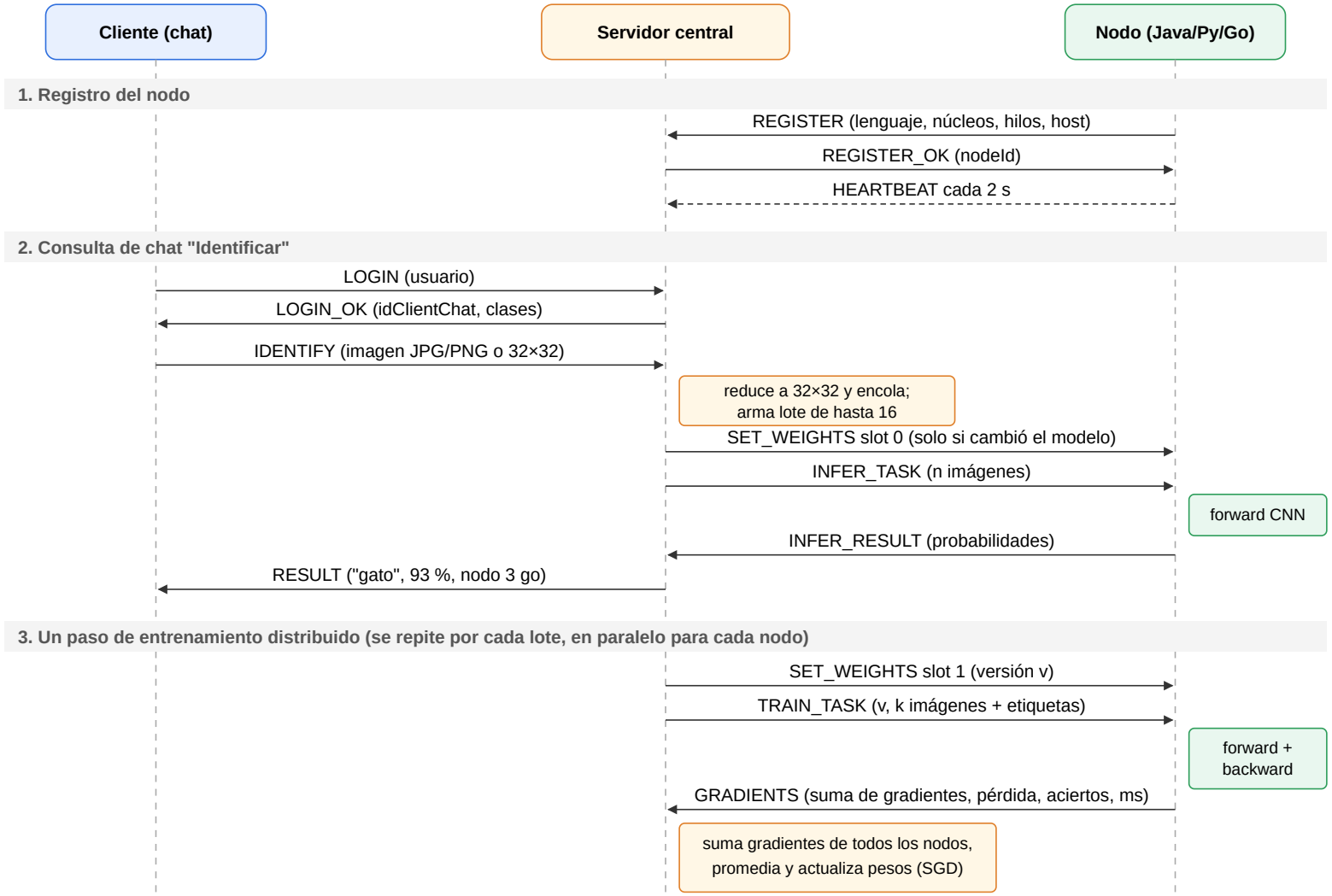
Protocolo OC/1 (sockets TCP, big-endian)

```
| 'O' | 'C' | versión | tipo | id petición (int32) | longitud (int32) | payload |
```

Nodo ↔ servidor (:6000)	
REGISTER / OK	lenguaje, núcleos, hilos
HEARTBEAT	cada 2 s
SET_WEIGHTS	slot, versión, pesos
TRAIN_TASK → GRADIENTS	suma de gradientes
INFER_TASK → RESULT	probabilidades

Cliente ↔ servidor (:5000)	
LOGIN / OK	idClientChat, clases
IDENTIFY → RESULT	"gato", 93 %, nodo
HISTORY, STATUS	historial, estado
TRAIN_REQ, CROSSCHECK	administración
ERROR	con el id de la petición

Diagrama de protocolo

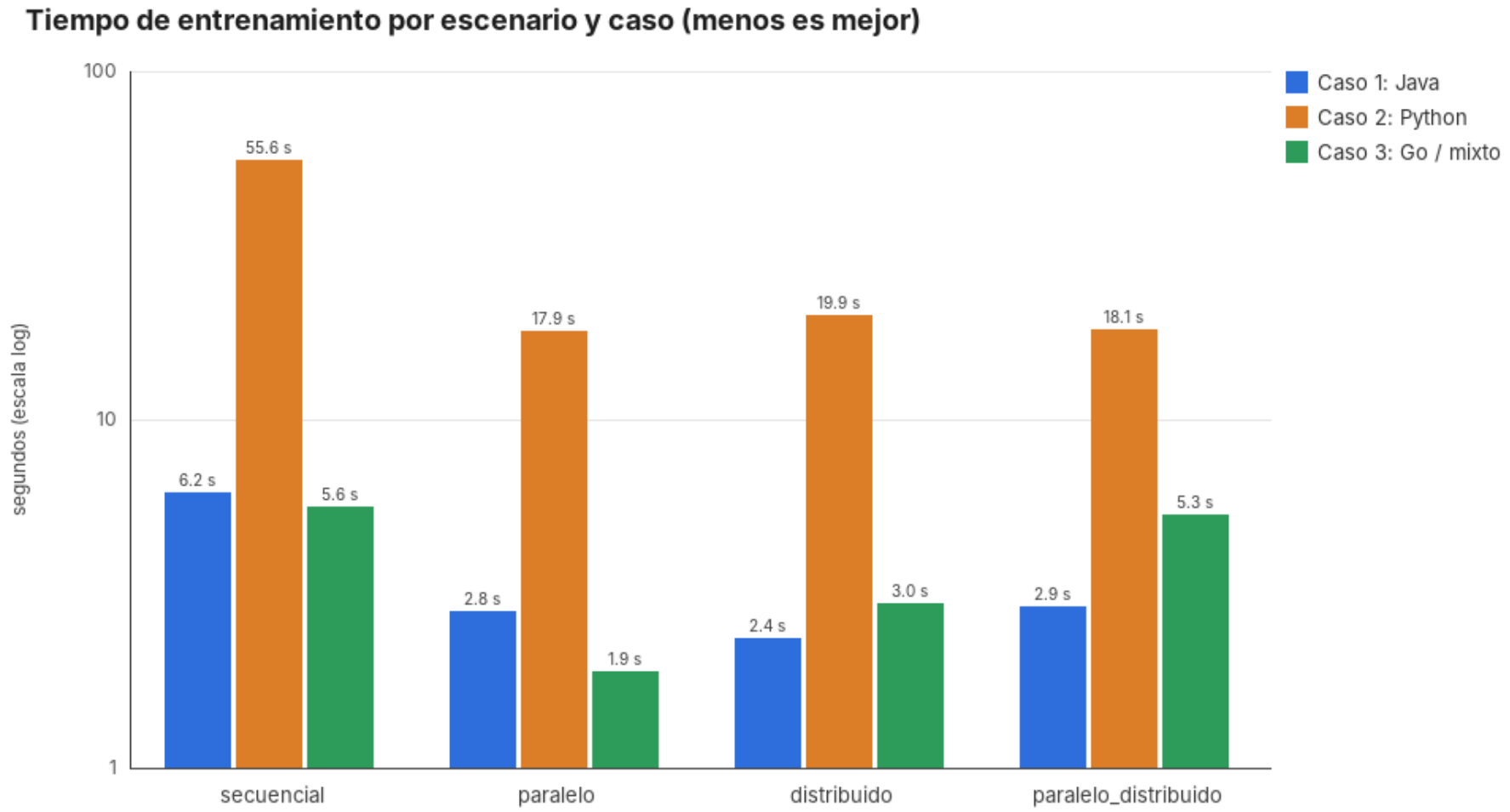


Si un nodo cae (socket cerrado o 10 s sin heartbeat), su parte del lote se reenvía a otro nodo vivo.

Correctitud y tolerancia a fallos

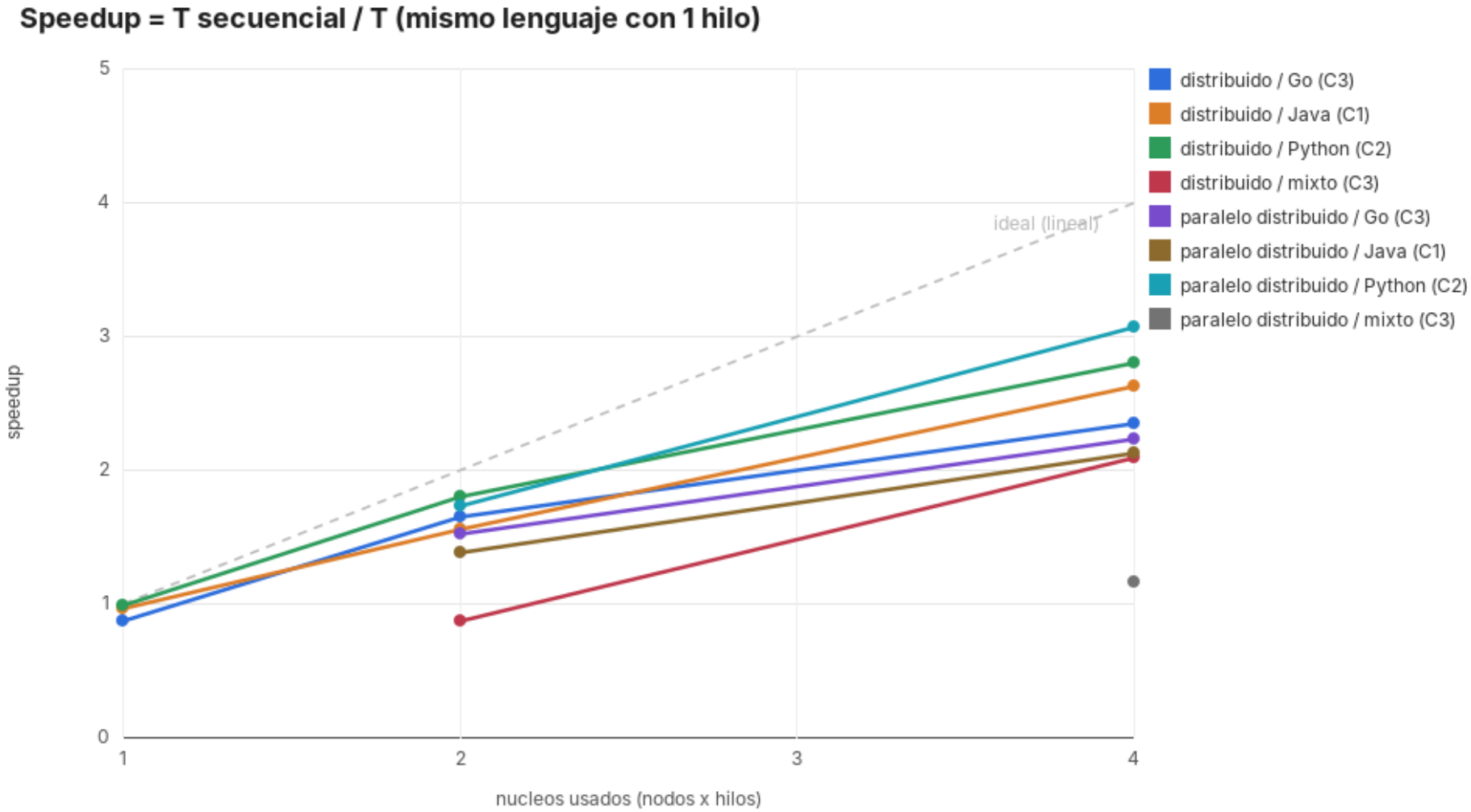
- **Gradiente numérico** coincide con el analítico en todas las capas
- **Prueba cruzada**: mismas 8 imágenes a cada nodo → **OK en Java, Python y Go** (Go y Java idénticos bit a bit)
- **Distribuido = local**: misma pérdida y acierto en 1 proceso o en el clúster
- Nodo caído (socket cerrado o 10 s sin heartbeat) → su parte se **reasigna**
- Los nodos se reconectan solos; cada tarea lleva la **versión** de los pesos
- Cola acotada: si se llena responde "servidor saturado"

Matriz de 12 casos

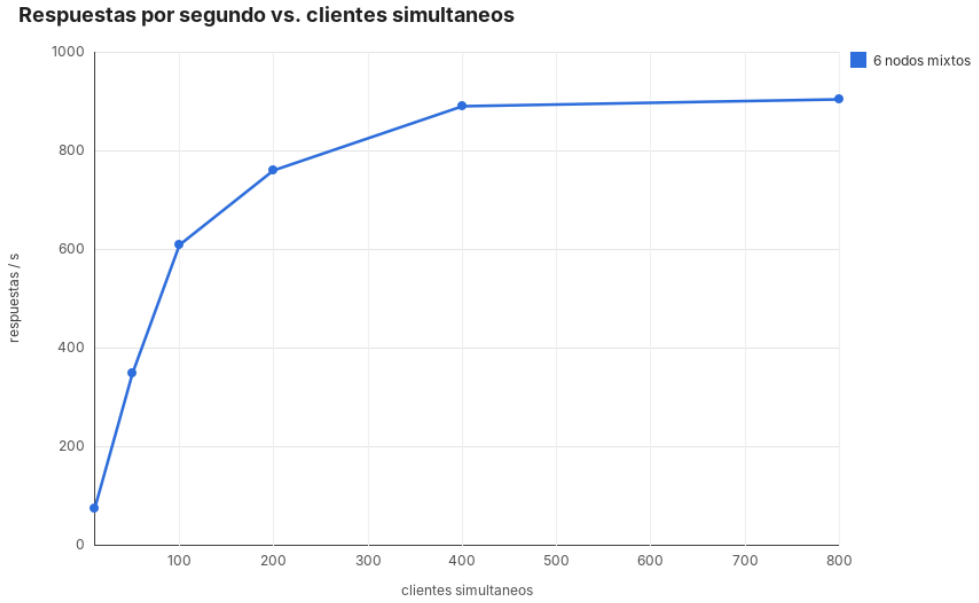


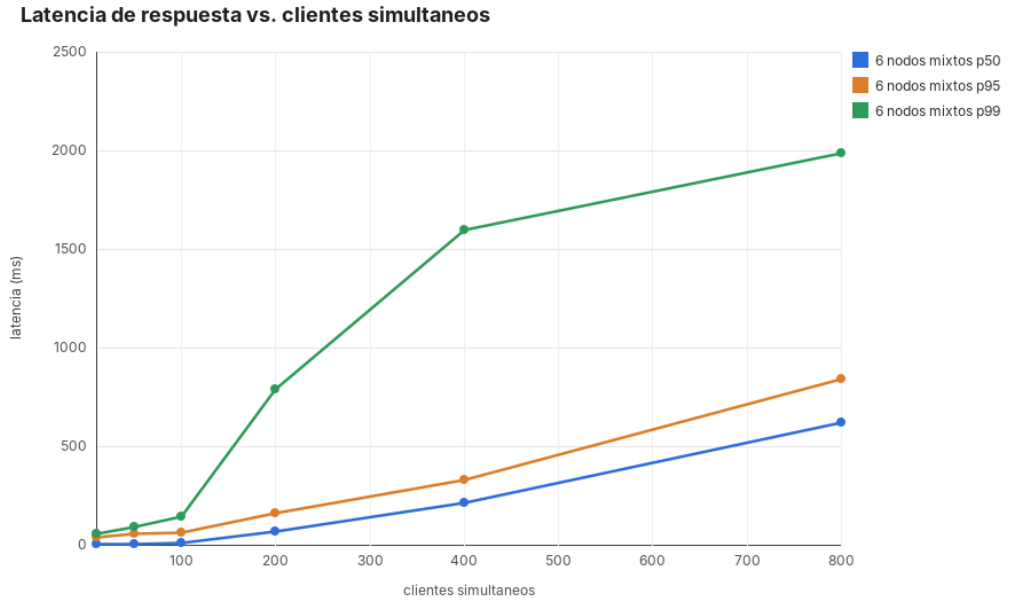
Medido en 1 máquina de 4 núcleos (los nodos comparten CPU). 1 024 imágenes, 1 época, lote 64. Se repite en el laboratorio con los mismos scripts.

Escalabilidad con el número de nodos



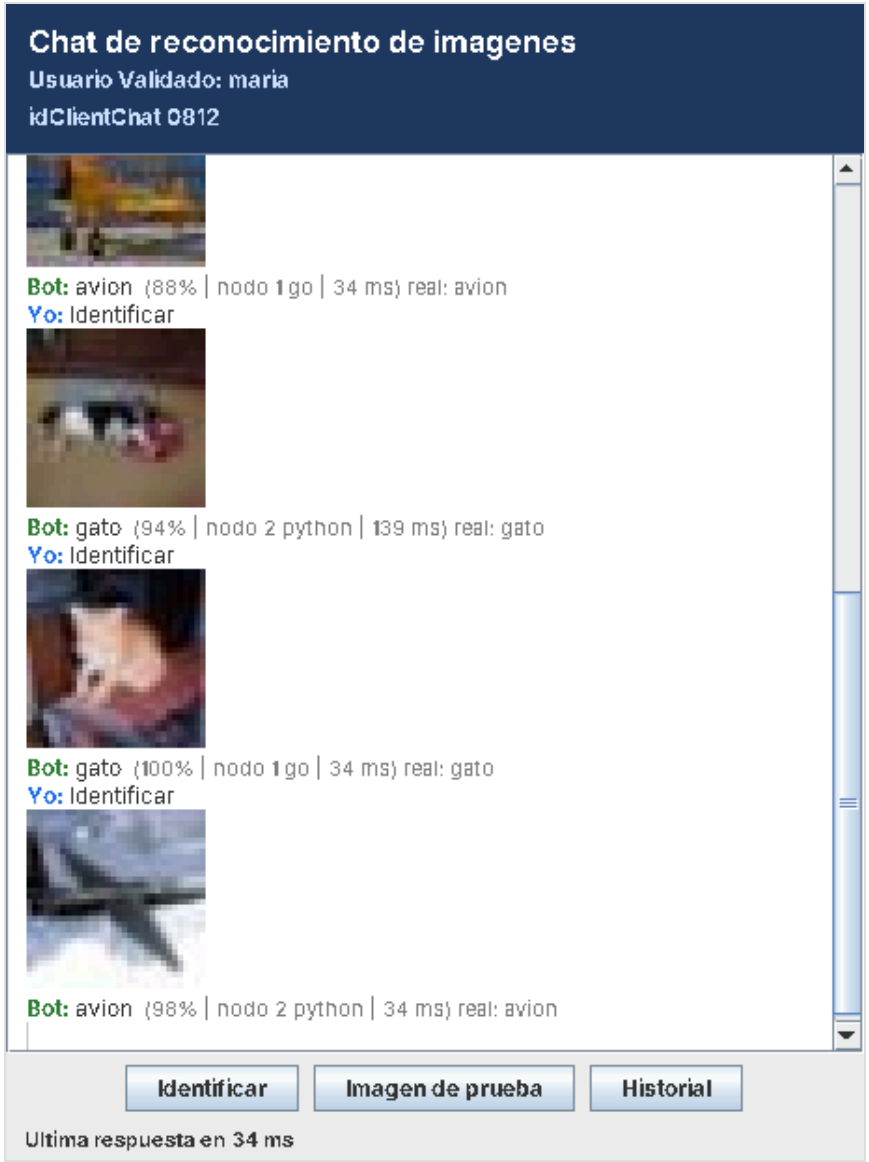
Pruebas de carga: ¿dónde se satura?





- 31 200 consultas, **0 errores**, hasta 800 clientes simultáneos con delay aleatorio de 0 a 200 ms
- Se satura en **≈900 resp/s**; después solo sube la latencia (mediana 624 ms con 800 clientes)

Demo: el chat



- Login → **idClientChat**
- "Identificar" → elige JPG/PNG
- El servidor la reduce a 32×32
- La calcula el nodo libre
- "Bot: gato (93 % · nodo 3 go)"
- Historial guardado en el servidor

Conclusiones

- Sistema completo con solo la librería estándar: CNN, servidor, 3 tipos de nodo, chat y carga
- Java, Python y Go calculan lo mismo (verificado)
- Hasta **3×** más rápido en 4 núcleos; límites: parte secuencial, comunicación y nodo más lento
- ≈900 respuestas/s y 0 errores con 800 clientes
- Python simple pero 10× más lento sin numpy; Go, el mejor para nodos

¿Preguntas?

Open AI Cubes · Java · Python · Go · sockets e hilos